

GUARDIAN CARE AUTONOMOUS SAFETY NETWORK

1.INTRODUCTION

In today's rapidly evolving, technology-driven world, ensuring the safety and protection of restricted and sensitive areas has become more important than ever. Facilities such as laboratories, data centre's, server rooms, warehouses, research institutions, industrial plants, and private properties store valuable equipment, confidential information, and critical infrastructure. Any unauthorized access or security breach can result in significant financial loss, operational disruption, or compromise of sensitive data.

Traditional security systems, including manual surveillance, security guards, and standalone CCTV cameras, have long been used to monitor such areas. However, these systems often rely heavily on continuous human supervision, which can lead to delayed responses, human error, and increased operational costs. Moreover, conventional systems may not provide intelligent alerts or real-time remote access, limiting their overall effectiveness in modern security environments.

The **GUARDIAN CARE AUTONOMOUS SAFETY NETWORK** is developed to address these challenges by integrating Internet of Things (IoT) technology into a smart and automated security framework. The core of the system is built around Passive Infrared (PIR) sensors, which detect motion by sensing changes in infrared radiation within the monitored area. When movement is detected in a restricted zone, a Wi-Fi-enabled microcontroller—such as the ESP8266 instantly processes the signal and transmits real-time alerts to authorized personnel through internet connectivity.

These notifications can be delivered via mobile applications, SMS, email alerts, or web-based dashboards, ensuring immediate awareness of potential threats. This automated response mechanism significantly reduces dependency on constant on-site monitoring and enables faster decision-making during critical situations.

One of the major strengths of the system is its remote accessibility and centralized monitoring capability. Users can securely log in to a cloud-based platform to view live activity status, receive instant notifications, and access historical logs for analysis and reporting. The system is designed to be scalable, making it suitable for both small-scale installations and large industrial environments.

1.1. ORGANISATION PROFILE

NOBEL SOFTWARE SOLUTIONS is a professional website designing, Customized software development, Business process Outsourcing – IT/ITES & Internet marketing company providing full featured web service including B2A Acquisition & B2C e-commerce solutions and acting as an offshore development center for overseas development firms. NOBEL SOFTWARE SOLUTIONS is an innovative company, based in India that provides a series of Web-based and software and software applications that have helped their customers to create successful business ventures through online initiatives. NOBEL SOFTWARE SOLUTIONS provide all the services that a company needs to get online from web designing to web hosting and manage leading-edge web sites and e-business software solutions success in the domestic market. Ever since its inception, NOBEL software solutions has accrued continuous growth in all its business functions and this has been possible only due to its commitment, quality training methodologies, the services it offers, knowledge sharing with industry leaders and professional approach.

Services

- Professional Web Design
- SEO concerts, Internet Marketing
- Pay per click Campaign
- Link Building, Ecommerce Solution
- Web Application Development
- Multimedia Presentations
- Customized Software development
- Business Process Outsourcing-IT/IT.

1.2.SYSTEM SPECIFICATION

1.2.1 HARDWARE REQUIREMENTS

For IoT Device Setup:

- Microcontroller : ESP8266 (NodeMCU)
- Sensor : PIR (Passive Infrared) Sensor
- Alert System : Buzzer / LED Indicator
- Power Supply : 5V DC Adapter / USB Power
- Connectivity : Wi-Fi Network
- Additional Components : Jumper Wires, Breadboard / PCB

For Monitoring System:

- Processor : Intel Dual Core or above
- Processor Speed : 2.0 GHz or higher
- RAM : 4 GB RAM (Minimum 2 GB)
- Hard Disk : 250 GB or above
- Monitor : 15-inch Color Monitor or higher
- Input Devices : Standard Keyboard and Scroll Mouse
- Smartphone : Android / iOS device with internet access (for Blynk App monitoring)

1.2.2.SOFTWARE SPECIFICATION

- Front end : Blynk App
- Back end : NodeMCU ESP8266 Programmable board
- Programming Language : Embedded C / Arduino IDE
- IoT Platform : Blynk IoT Platform
- Microcontroller IDE : Arduino IDE
- Operating System : Windows 10 or above
- Mobile Application : Blynk App (Android / iOS)
- Browser (Optional) : Google Chrome / Edge for Blynk Web Dashboard

1.3.SOFTWARE DESCRIPTION

1.3.1.INTRODUCTION TO FRONT END

PYTHON:

Python is a high-level, interpreted and general-purpose dynamic programming language that focuses on code readability. The syntax in Python helps the programmers to do coding in fewer steps as compared to Java or C. Python is widely used in bigger organizations because of its multiple programming paradigms. They usually involve imperative and object-oriented functional programming. It has a comprehensive and large standard library that has automatic memory management and dynamic features.

The software development companies prefer Python language because of its versatile features and fewer programming codes. Nearly 14% of the programmers use it on the operating systems like UNIX, Linux, Windows and Mac OS. Interactive

Advantages or Benefits of Python

The Python language has diversified application in the software development companies such as in gaming, web frameworks and applications, language development, prototyping, graphic design applications, etc. This provides the language a higher plethora over other programming languages used in the industry. Some of its advantages are-

Extensive Support Libraries

It provides large standard libraries that include the areas like string operations,Internet, web service tools, operating system interfaces and protocols. Most of the highly used programming tasks are already scripted into it that limits the length of the codes to be written in Python.

Presence of Third Party Modules

The Python Package Index (PyPI) contains numerous third-party modules that make Python capable of interacting with most of the other languages and platforms.

Integration Feature

Python integrates the Enterprise Application Integration that makes it easy to develop Web services by invoking COM or COBRA components. It has powerful control capabilities as it calls directly through C, C++ or Java via Jython. Python also processes XML and other markup languages as it can run on all modern operating systems through same byte code.

Improved Programmer's Productivity

The language has extensive support libraries and clean object-oriented designs that increase two to ten fold of programmer's productivity while using the languages like Java, VB, Perl, C, C++ and C#.

Productivity

With its strong process integration features, unit testing framework and enhanced control capabilities contribute towards the increased speed for most applications and productivity of applications. It is a great option for building scalable multi-protocol network applications.

Open Source and Community Development

Python language is developed under an OSI-approved open source license, which makes it free to use and distribute, including for commercial purposes. Further, its development is driven by the community which collaborates for its code through hosting conferences and mailing lists, and provides for its numerous modules.

Learning Ease and Support Available

Python offers excellent readability and uncluttered simple-to-learn syntax which helps beginners to utilize this programming language. The code style guidelines, PEP 8, provide a set of rules to facilitate the formatting of code. Additionally, the wide base of users and active developers has

resulted in a rich internet resource bank to encourage development and the continued adoption of the language.

User-friendly Data Structures

Python has built-in list and dictionary data structures which can be used to construct fast runtime data structures. Further, Python also provides the option of dynamic high-level data typing which reduces the length of support code that is needed.

Simple and easy to Learn

1. The syntaxes of python language are very simple. Anybody can remember the python language syntaxes, rules and regulations very easily.
2. The elegant syntaxes of the python language makes the people to learn python in easiest manner.
3. With out having any other programming languages knowledge, we can learn python directly.
4. The simple and powerful syntax of python language makes the programmers to express their business logic is less lines of code.

Platform Independent

1. Like Java programs, Python programs are also platform independent.
2. Once we write a Python program, it can run on any platform without rewriting once again.
3. Python uses PVM to convert python code to machine understandable code

High Level Language

1. Python is a high level language.
2. While developing python applications, developers no need to bother about memory management.
3. Dynamically Typed Language
4. Python is a dynamically typed language. I mean, in python there is no need to declare

type of a variable.

5. Whenever we assign a value to the variable, based on the value the type will be allocated automatically.

Python supports POP &OOP

1. Python language supports both Procedural Oriented Programming and Object Oriented Programming features.
2. We can implement OOPs features like Encapsulation, polymorphic, inheritance and abstraction in python programming language.

Python is interpreted Language

1. Like PHP, Python is also interpreted language.
2. Python applications do not require explicit compilation so that compiler is not require for python software.
3. Python interpreter is responsible for execution of python applications.
4. When ever we run a python application, the python interpreter checks for the syntax errors. If there is no syntax errors found in our code then the interpreter converts the code into intermediate code in the form of low level format and executes it.
5. The intermediate code of the python applications is known as byte code.
6. The extension of the byte code file is .pyc (Python Compile Code).

Python is Embeddable :

1. We can embed the python code into other languages such as C,C++, Java and Etc..
2. In order to provide the scripting capabilities to other languages we can use the python code in it.

Develop GUI & Web Application :

1. We can develop the GUI based applications using Python Languages.
2. We can also develop the Web applications using Python Language.

Applications of Python:

- GUI based desktop applications
- Image processing and graphic design applications
- Scientific and computational applications
- Games
- Web frameworks and web applications
- Enterprise and business applications
- Operating systems
- Language development
- Prototyping

2.SYSTEM STUDY

2.1. EXISTING SYSTEM

In the existing system, security in restricted areas is mainly maintained using traditional methods such as manual guarding, standalone CCTV cameras, and basic alarm systems. Security personnel are assigned to monitor sensitive locations, while CCTV cameras continuously record footage for later review. Some systems use simple motion detectors connected to local alarms, which only produce sound alerts within the premises. However, these systems do not provide real-time remote notifications or smart monitoring capabilities. Monitoring usually requires continuous human supervision, and recorded footage must be manually checked to identify security breaches. As a result, response time may be delayed, and unauthorized access might go unnoticed until after the incident has occurred.

2.1.1. DRAWBACKS

- Requires continuous human monitoring and supervision.
- No real-time remote alerts to authorized users.
- Delayed response to unauthorized access.
- High installation and maintenance cost for CCTV systems.
- Limited storage and manual checking of recorded footage.
- Lack of smart integration with mobile or cloud platforms.

2.2. PROPOSED SYSTEM

The proposed system, GUARDIAN CARE AUTONOMOUS SAFETY NETWORK -Motion Detection System for Restricted Areas, is an intelligent and automated security solution designed using ESP8266 and IoT technology. The system uses a PIR motion sensor to detect

human movement in restricted zones. When motion is detected, the ESP8266 module processes the signal and instantly sends real-time alerts to authorized users through the Blynk mobile application over Wi-Fi. The system can also activate a buzzer or LED indicator as an immediate local alert. All activity notifications can be monitored remotely using a smartphone, ensuring quick response to security threats. This IoT-based approach reduces human dependency and enhances overall surveillance efficiency and reliability.

2.2.1. ADVANTAGES OF PROPOSED SYSTEM

- Provides real-time motion detection alerts.
- Enables remote monitoring through smartphone access.
- Reduces dependency on manual surveillance.
- Activates instant local alarms (buzzer/LED).
- Cost-effective and affordable implementation.
- Energy-efficient operation.
- Easy to install and maintain.
- Scalable and expandable for future upgrades.

2.2.2. FEATURES

- Real-time motion detection using PIR sensor.
- Instant alert notifications through the Blynk app.
- Remote monitoring via smartphone.
- Wi-Fi enabled ESP8266 connectivity.
- Local alarm activation using buzzer or LED.
- Cost-effective and scalable security solution.

3.SYSTEM DESIGN AND DEVELOPMENT

3.1. System Design

- **File Design:**

The file design of the system organizes all project resources into a well-structured directory hierarchy to ensure easy maintenance and scalability. The main application file (app.py) controls routing, request handling, and interaction with the Stable Diffusion model. Template files stored inside the templates folder define the user interface layouts, while static resources such as CSS stylesheets, uploaded images, and generated results are stored inside the static directory. Model-related files such as the Stable Diffusion .safetensors file and CLIP model files are placed inside a separate models folder to clearly separate application logic from large model data.

The system also includes a local SQLite database file (database.db) that stores generation metadata such as prompts, settings, and file paths of output images. Uploaded images are saved inside static/uploads, and generated images are saved inside static/results. This file organization improves access speed, simplifies debugging, and ensures that different components of the system do not interfere with each other.

- **Input Design:**

The input design focuses on collecting user data in a structured and user-friendly manner through HTML forms. The main inputs include text prompt, style selection, lighting selection, camera view, background description, negative prompt, generation steps, and guidance scale. Dropdown menus and sliders are used for parameter selection to reduce user errors. All inputs are validated on the server side before processing to ensure correct data types and acceptable ranges.

For Image-to-Image generation, an additional file input field allows users to upload a base image. The system checks the file format, size, and validity before saving it to the server. This validation process prevents invalid or malicious files from being processed and ensures stable system operation.

- **Output Design:**

The output design presents the generated images clearly on the web interface after processing. Each generated image is displayed along with options to download the image. The system also shows generation details such as prompt, selected parameters, and generation time, which helps users understand how the output was produced.

In addition to real-time display, the system stores all generated image paths and metadata in the SQLite database. A history page allows users to view previous generations, revisit settings, and delete unwanted records. This improves usability and enables users to track their work efficiently.

- **Code Design:**

The code design follows a modular and layered approach. Separate functions are used for database handling, input validation, model loading, and image generation. This improves readability and makes the system easier to maintain and extend. The Stable Diffusion model is loaded once and reused for multiple requests to reduce overhead.

Error handling mechanisms are implemented using try-except blocks to capture runtime issues such as model loading failures or invalid inputs. The application uses Flask routing to manage different pages such as generation, history, and result view. This structured code design ensures reliability, reusability, and scalability of the system.

3.4. SYSTEM DEVELOPMENT

3.4.1. DESCRIPTION OF MODULES

1. Motion Detection Module:

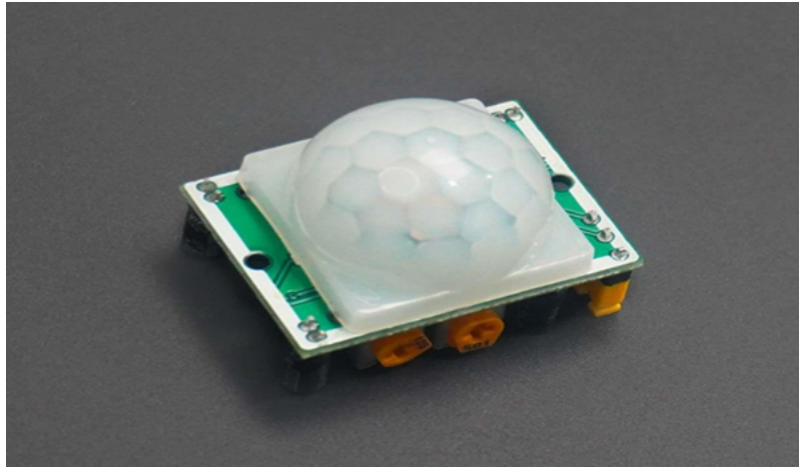


Fig 1: PIR Sensor

This module consists of a Passive Infrared (PIR) sensor that detects human movement within a restricted area. The sensor identifies changes in infrared radiation caused by body heat and sends a signal to the microcontroller when motion is detected. It serves as the primary input component of the system.

2. Microcontroller Module (ESP8266):

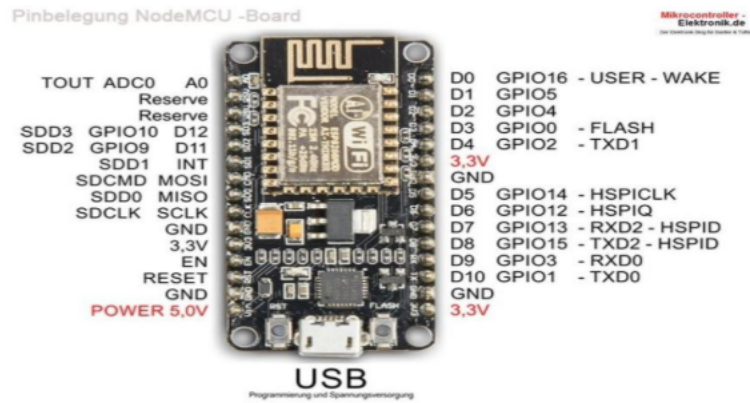


Fig 2: NodeMCU ESP8266

The ESP8266 (NodeMCU) acts as the central processing unit of the system. It receives signals from the PIR sensor, processes the data, and determines whether motion has occurred. The module also manages Wi-Fi connectivity and communicates with the Blynk IoT platform to send alerts.

3. Alert and Notification Module:

This module is responsible for generating alerts when motion is detected. It activates a buzzer or LED for local warning and sends instant notifications to authorized users through the Blynk mobile application. This ensures both on-site and remote alert mechanisms.

4. IoT Communication Module:

The IoT communication module enables real-time data transmission through Wi-Fi. Using the Blynk cloud platform, the system sends motion detection updates to smartphones or web dashboards, allowing users to monitor activity from anywhere with internet access.

5. User Interface Module:

The user interface is developed using the Blynk mobile application. It allows users to receive notifications, monitor system status, and control certain features such as enabling or disabling alerts. This module ensures easy interaction between the user and the system.

6. Power Supply Module:

This module provides stable power to the ESP8266 and connected components. It uses a 5V DC adapter or USB power source to ensure continuous and reliable system operation.

4.TESTING AND IMPLEMENTATION

4.1 TESTING

Testing is an important phase in the development of the IoT system using ESP8266 connected to the Blynk platform. The testing process ensures that both hardware and software components function correctly and communicate reliably over the internet. Initially, individual components such as the ESP8266 module, sensors, and output devices (LED, buzzer, etc.) were tested separately to verify proper connections and functionality. Sensor readings were checked to confirm accurate data acquisition. Next, Wi-Fi connectivity testing was performed to ensure that the ESP8266 could successfully connect to a stable wireless network.

The communication between the ESP8266 and the Blynk cloud server was verified by monitoring real-time data updates on the Blynk mobile application. Notification testing was also conducted to confirm that alerts were received instantly when triggered.

Integration testing was carried out to validate the complete system workflow, ensuring smooth interaction between hardware, cloud platform, and mobile interface. Finally, reliability and performance testing were performed by running the system continuously to check stability, response time, and error handling under different conditions.

THE OBJECTIVE OF TESTING IS AS FOLLOWS:

- To verify proper communication between the ESP8266 module and the Blynk cloud platform over Wi-Fi.
- To ensure accurate sensor data transmission and reliable real-time alerts through the Blynk mobile application

TESTING ISSUES

- The ESP8266 may face unstable or weak internet connection, causing delays or failure in sending data to the Blynk server.
- Insufficient or unstable power can lead to unexpected restarts or malfunction of the ESP8266 module.
- Incorrect wiring or improper calibration may result in inaccurate sensor readings or false triggers.
- Network latency or Blynk server downtime may cause delayed notifications or temporary data synchronization issues.

4.1.1 TESTING METHODOLOGIES

Testing methodologies for the IoT system using ESP8266 connected to the Blynk platform follow a systematic approach to ensure reliability, accuracy, and performance. The first methodology is Unit Testing, where individual components such as the ESP8266 module, sensors, relays, LEDs, and buzzer are tested independently to verify proper functionality. Each hardware component is checked for correct wiring and response.

The second methodology is Connectivity Testing, which ensures that the ESP8266 successfully connects to the Wi-Fi network and communicates with the Blynk cloud server. Data transmission between the device and the mobile application is monitored for consistency and speed.

The third methodology is Integration Testing, where all hardware and software components are combined and tested as a complete system. This verifies smooth interaction between sensors, microcontroller, cloud platform, and user interface. Finally, Performance and Reliability Testing is conducted by running the system continuously under different network conditions to evaluate stability, response time, and error handling. These methodologies ensure the IoT system operates efficiently and reliably in real-world environments.

4.2 TYPES OF TESTING

The different types of testing include:

- System Testing
- Unit Testing
- Integration Testing
- Validation Testing
- System Implementation

System Testing:

System testing is performed to evaluate the complete IoT system using ESP8266 connected to the Blynk platform as a fully integrated unit. In this phase, all hardware components such as sensors, actuators, and the ESP8266 module are connected and tested together with the software and cloud environment.

The main objective is to ensure that the entire system works according to the project requirements. During system testing, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk application using the authentication token. Sensor inputs are triggered to verify whether the data is correctly processed and transmitted to the Blynk dashboard in real time. Output devices such as LEDs or buzzers are tested to confirm proper activation based on commands or sensor values. Notification testing is also conducted to ensure alerts are received without delay.

The system is observed under continuous operation to check stability, response time, and error handling. This testing ensures the IoT solution functions reliably in real-time environments.

Unit Testing:

Unit testing is performed to verify the proper functioning of each individual component in the IoT system using ESP8266 connected to the Blynk platform. In this phase, every hardware and software unit is tested separately before integrating them into the complete system.

The ESP8266 module is first tested to ensure it powers on correctly and establishes a stable Wi-Fi connection. Basic programs such as LED blinking are uploaded to confirm proper board configuration and communication with the Arduino IDE. Next, individual sensors are tested to check whether they provide accurate readings and respond correctly to environmental changes. Output components such as LEDs, buzzers, or relays are also tested independently to ensure they activate when triggered. Additionally, Blynk authentication tokens and virtual pins are verified to confirm proper data transmission between the ESP8266 and the Blynk mobile application. By testing each module independently, errors can be identified and resolved early, ensuring smooth integration and reliable system performance in later stages.

Integration Testing:

Integration testing is performed to verify the proper interaction between all hardware and software components in the IoT system using ESP8266 connected to the Blynk platform. In this phase, individual modules such as sensors, actuators, ESP8266 microcontroller, Wi-Fi network, and the Blynk mobile application are combined and tested as a complete system.

The testing process begins by connecting sensors and output devices to the ESP8266 and uploading the integrated program. Once powered on, the ESP8266 is connected to a Wi-Fi network and linked to the Blynk cloud server using the authentication token. Sensor data is monitored in real-time on the Blynk application to confirm successful communication between the hardware and cloud platform. Alerts, notifications, and device control features are tested to ensure proper synchronization. Any delay in data transmission or mismatch in readings is identified and corrected. Integration testing ensures seamless coordination among components and confirms that the system operates reliably in real-world IoT environments.

Validation Testing:

Validation testing is performed to ensure that the IoT system using ESP8266 connected to the Blynk platform meets the intended project requirements and functions as expected in real-time conditions. This process verifies whether the complete system satisfies user needs and operational objectives.

Initially, the system is tested by triggering input devices such as sensors or switches to confirm that the ESP8266 correctly processes the input data. The output responses, such as LED indication, buzzer alerts, or relay activation, are observed to validate proper hardware functionality. Simultaneously, the Blynk mobile application is monitored to ensure that real-time data updates and notifications are received without delay. Further validation includes testing the system under different network conditions to confirm stable Wi-Fi connectivity and reliable cloud communication. Multiple test cases are executed to verify accuracy, response time, and consistency. Continuous operation testing is also conducted to ensure long-term stability. Through validation testing, the system's performance, reliability, and user requirements compliance are successfully confirmed.

System Implementation:

System implementation testing is carried out after completing the development and integration of the IoT system using ESP8266 with the Blynk platform. This phase ensures that the fully implemented system operates correctly in a real-time environment. Initially, the hardware components such as sensors, ESP8266 module, and output devices are assembled and verified for proper physical connections.

The firmware is then uploaded to the ESP8266 using the Arduino IDE. Once deployed, the system is connected to a Wi-Fi network and linked to the Blynk cloud using the assigned authentication token.

Real-time testing is performed by triggering sensors and observing whether the corresponding data is accurately updated in the Blynk mobile application. Notifications and control commands are tested to confirm two-way communication between the device and the user interface.

The system is also monitored for stability by running it continuously to detect issues such as disconnections, delays, or hardware failures. This testing ensures the implemented IoT solution functions reliably under practical conditions.

5. CONCLUSION AND FUTURE ENHANCEMENT

5.1.Conclusion

The project “GUARDIAN CARE AUTONOMOUS SAFETY NETWORK – Motion Detection System for Restricted Areas” successfully demonstrates an efficient and intelligent security solution using ESP8266 and IoT technology. The system effectively detects motion in restricted zones using a PIR sensor and provides instant real-time alerts through the Blynk mobile application. By integrating Wi-Fi connectivity and cloud communication, the system enables remote monitoring and quick response to unauthorized access.

It reduces dependency on manual surveillance and enhances security through automation. The solution is cost-effective, scalable, and easy to implement in various environments such as offices, laboratories, and warehouses. Overall, the project proves that IoT-based motion detection systems can significantly improve safety, reliability, and monitoring efficiency in restricted areas.

5.2.Future Enhancement

The GUARDIAN CARE AUTONOMOUS SAFETY NETWORK– Motion Detection System for Restricted Areas can be further enhanced by integrating advanced technologies to improve security and functionality. In the future, a camera module can be added to capture images or record video when motion is detected, providing visual evidence of unauthorized access. Face recognition technology can also be implemented to identify authorized and unauthorized individuals. Integration with cloud storage can help maintain detailed activity logs for long-term analysis. The system can be expanded with additional sensors such as smoke, gas, or door sensors for comprehensive security monitoring. Furthermore, implementing GSM or SMS alert features can ensure notifications even without Wi-Fi connectivity. These enhancements will make the system more intelligent, reliable, and versatile.

BIBLIOGRAPHY

Reference books

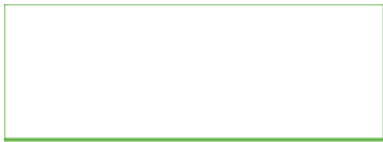
1. Banzi, M., & Shiloh, M. (2014). Getting Started with Arduino. Maker Media.
2. Monk, S. (2016). Programming Arduino: Getting Started with Sketches. McGraw-Hill Education.
3. Blum, J. (2019). Exploring Arduino: Tools and Techniques for Engineering Wizardry. Wiley.
4. Kolban, N. (2018). Kolban's Book on ESP8266. Leanpub.
5. Richardson, M., & Wallace, S. (2012). Getting Started with Raspberry Pi. O'Reilly Media.
6. McEwen, A., & Cassimally, H. (2014). Designing the Internet of Things. Wiley.
7. Bahga, A., & Madiseti, V. (2015). Internet of Things: A Hands-On Approach. Universities Press.
8. Minoli, D. (2017). Building the Internet of Things with IPv6 and MIPv6. Wiley.
9. Reese, G. (2018). Cloud Application Architectures. O'Reilly Media.
10. Monk, S. (2018). Arduino IoT Cloud for Developers. Packt Publishing.
11. Wilmshurst, T. (2010). Designing Embedded Systems with PIC Microcontrollers. Newnes.

WEBSITE REFERENCES

- <https://blynk.io>
- <https://www.espressif.com/en/products/socs/esp8266>
- <https://www.arduino.cc>
- <https://randomnerdtutorials.com>
- <https://www.iotforall.com>

APPENDIX

Symbols Of the Data Flow Diagram



Rectangle shape

Entity



Rounded Rectangle

Process



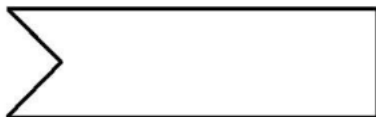
Arrow Head

Data Flow



Open Rectangle

Data Store

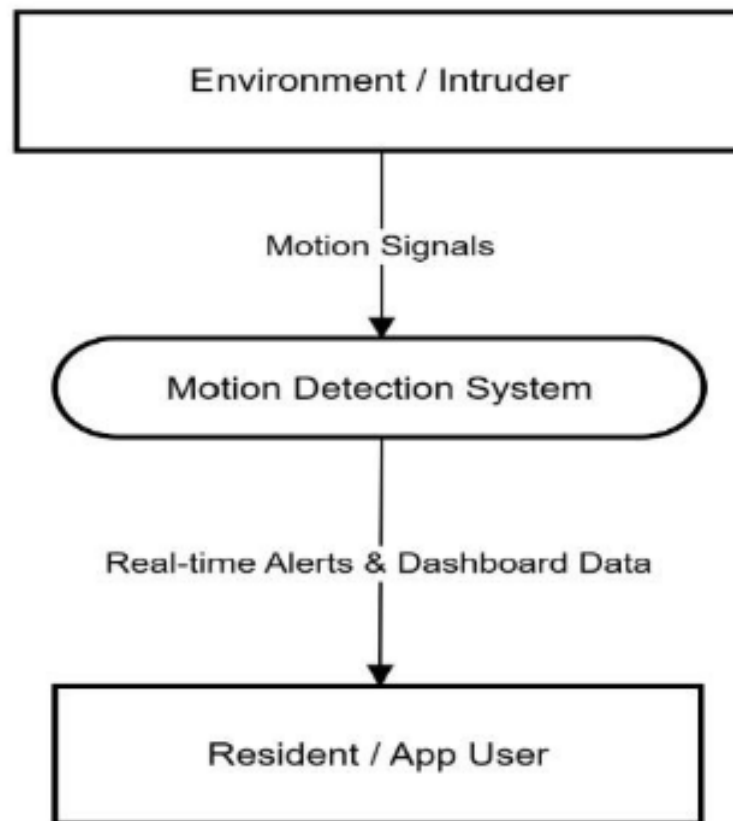


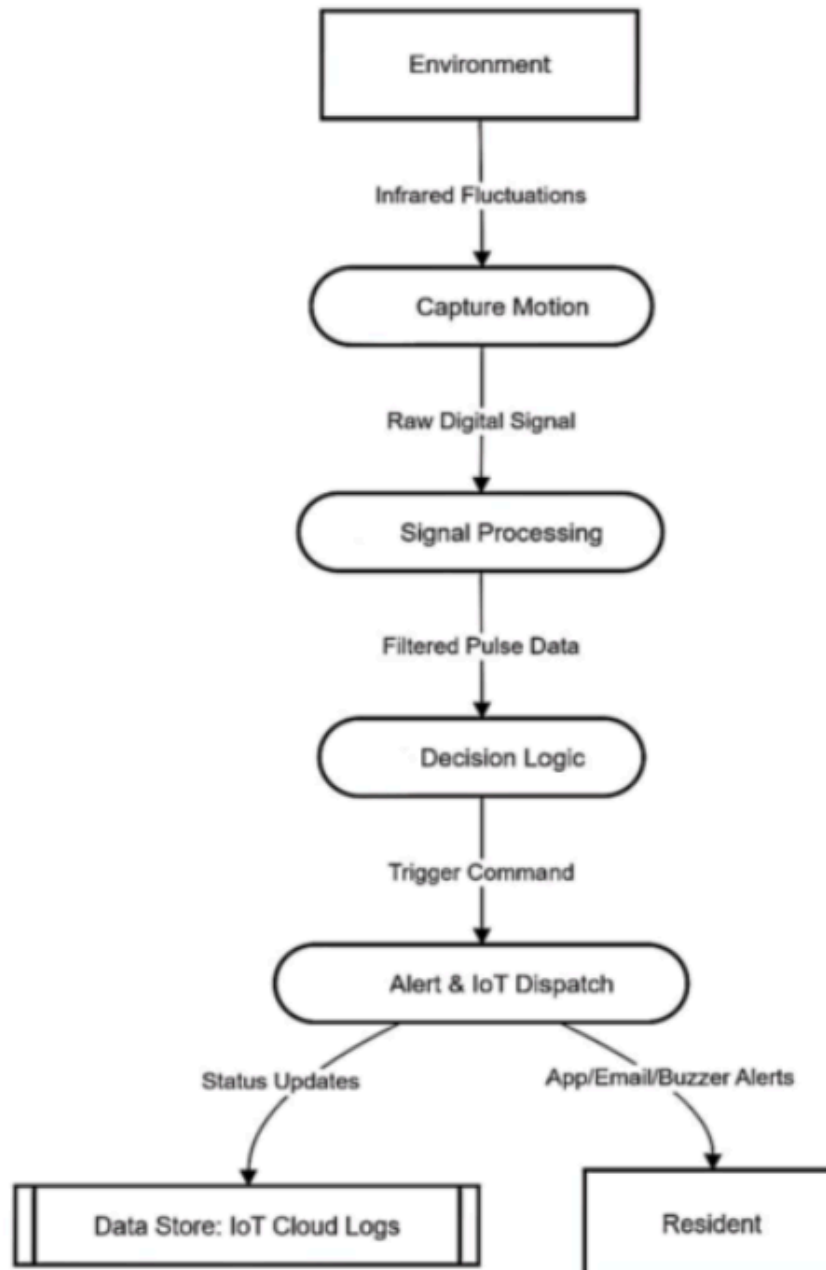
Curved Edge Button

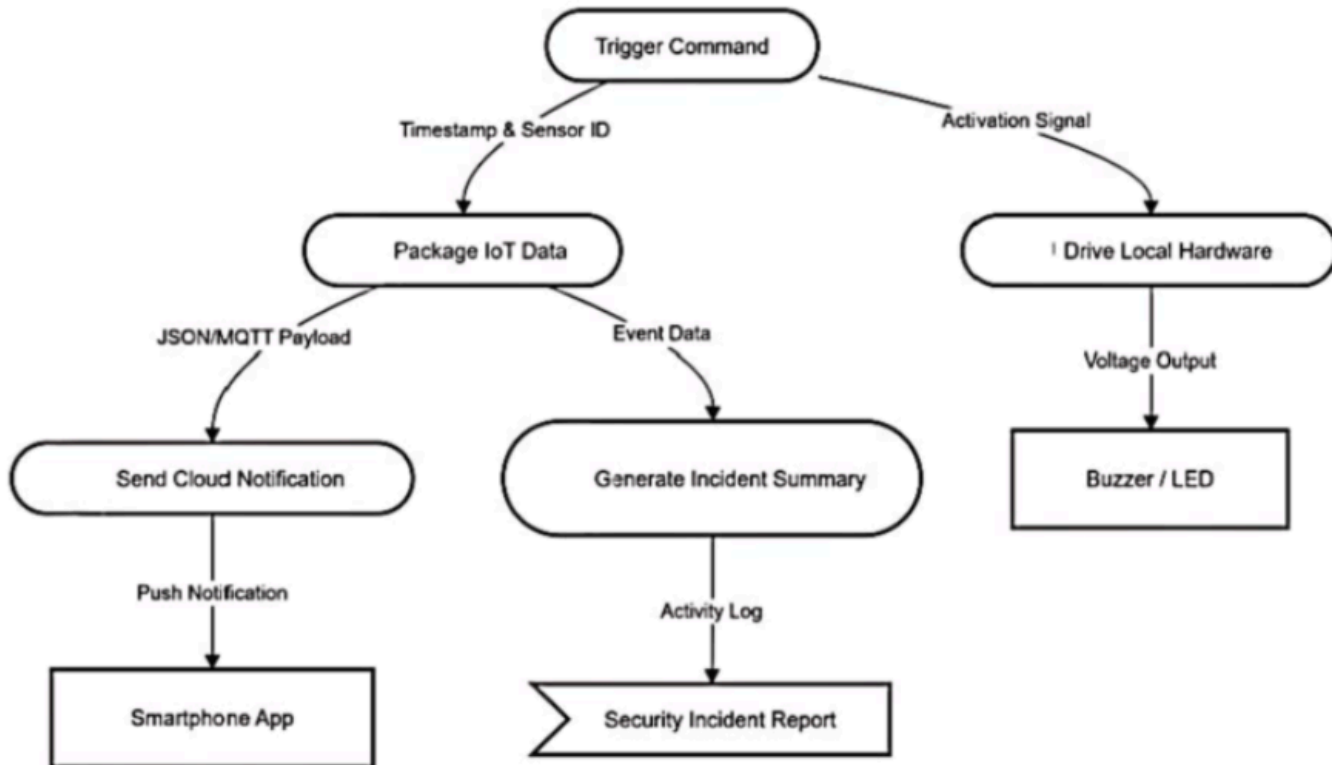
Reports

A. Data Flow Diagram(DFD)

LEVEL 0 :



LEVEL 1 :

LEVEL 2 :

B. WORKING DIAGRAM:

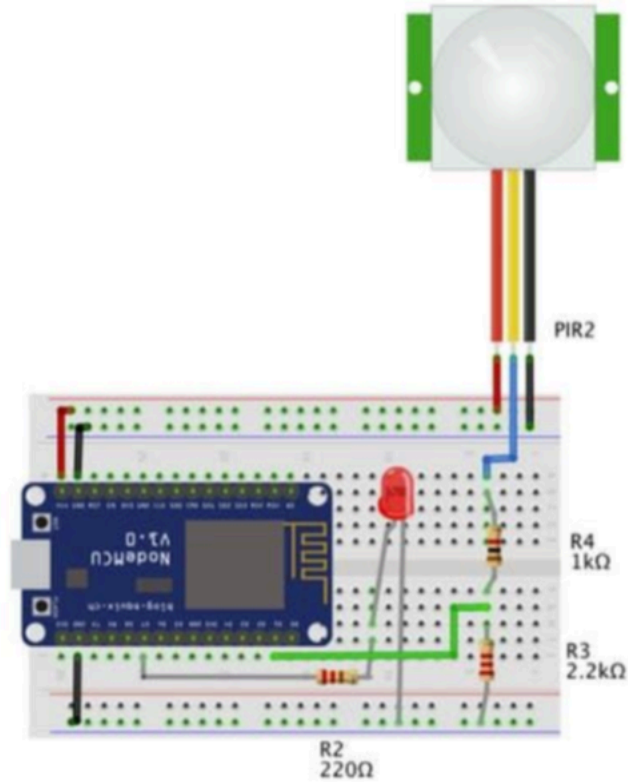


Fig 3: The working diagram shows how the PIR sensor is connected with the NodeMCU controller and LED to detect motion and process the signal for monitoring or alert generation.

C. Sample Source code:

```
# =====  
  
# ENTERPRISE IoT INTRUSION DETECTION SYSTEM  
  
# Device: ESP8266  
  
# Firmware Version: 2.0.0  
  
# Architecture: Modular Enterprise Design  
  
# Author: Prasanna Security Systems  
  
# =====  
  
# ===== IMPORTS =====  
  
import network  
  
import time  
  
import machine  
  
import urequests  
  
import ujson  
  
import gc  
  
import os  
  
from machine import Pin, Timer, WDT  
  
# ===== CONSTANTS =====  
  
FIRMWARE_VERSION = "2.0.0";  
  
DEVICE_NAME = "ESP8266_INTRUSION_NODE";
```

DEBUG_MODE = True

CONFIG_FILE = "config.json";

LOG_FILE = "events.log";

BLYNK_AUTH = "YOUR_BLYNK_AUTH_TOKEN";

SSID = "YOUR_WIFI_NAME";

PASSWORD = "YOUR_WIFI_PASSWORD";

PIR_PIN = 14

BUZZER_PIN = 12

LED_PIN = 13

MOTION_COOLDOWN = 5

HEARTBEAT_INTERVAL = 30

STATUS_INTERVAL = 10

WIFI_CHECK_INTERVAL = 15

MEMORY_CHECK_INTERVAL = 20

MAX_LOG_LINES = 200

===== LOGGER =====

class Logger:

LEVEL_INFO = &;INFO&;

LEVEL_WARN = &;WARN&;

LEVEL_ERROR = &;ERROR&;

LEVEL_DEBUG = &;DEBUG&;

```
def __init__(self):

self.log_count = 0

def log(self, level, message):

timestamp = time.time()

formatted = &quot;[{}][{}] {}&quot;.format(timestamp, level, message)

if DEBUG_MODE:

print(formatted)

self._write_to_file(formatted)

def _write_to_file(self, line):

try:

with open(LOG_FILE, &quot;a&quot;) as f:

f.write(line + &quot;\n&quot;)

except:

print(&quot;Log write failed&quot;)

def rotate_logs(self):

try:

size = os.stat(LOG_FILE)[6]

if size &gt; 10000:

os.remove(LOG_FILE)

self.log(self.LEVEL_INFO, &quot;Log rotated&quot;)

except:

pass
```

```
logger = Logger()
```

```
# ===== CONFIG MANAGER =====
```

```
class ConfigManager:
```

```
    def __init__(self):
```

```
        self.data = {
```

```
            "system_armed": True,
```

```
            "silent_mode": False,
```

```
            "motion_count": 0
```

```
        }
```

```
    def load(self):
```

```
        try:
```

```
            with open(CONFIG_FILE, "r") as f:
```

```
                self.data = ujson.load(f)
```

```
            logger.log(Logger.LEVEL_INFO, "Config loaded")
```

```
        except:
```

```
            logger.log(Logger.LEVEL_WARN, "No config found, using defaults")
```

```
            self.save()
```

```
    def save(self):
```

```
        try:
```

```
            with open(CONFIG_FILE, "w") as f:
```

```
                ujson.dump(self.data, f)
```

```
logger.log(Logger.LEVEL_INFO, &quot;Config saved&quot;)\n\nexcept:\n\nlogger.log(Logger.LEVEL_ERROR, &quot;Config save failed&quot;)\n\nconfig = ConfigManager()\n\nconfig.load()\n\n# ===== WIFI MANAGER =====\n\nclass WiFiManager:\n\n    def __init__(self):\n\n        self.wlan = network.WLAN(network.STA_IF)\n\n        self.wlan.active(True)\n\n    def connect(self):\n\n        if not self.wlan.isconnected():\n\n            logger.log(Logger.LEVEL_INFO, &quot;Connecting WiFi...&quot;)\n\n            self.wlan.connect(SSID, PASSWORD)\n\n            timeout = 0\n\n            while not self.wlan.isconnected() and timeout < 20:\n\n                time.sleep(1)\n\n                timeout += 1\n\n            if self.wlan.isconnected():\n\n                logger.log(Logger.LEVEL_INFO, &quot;WiFi Connected&quot;)\n\n            else:\n\n                logger.log(Logger.LEVEL_ERROR, &quot;WiFi Failed&quot;)
```

```
def check(self):

if not self.wlan.isconnected():

logger.log(Logger.LEVEL_WARN, &quot;WiFi Lost&quot;)

self.connect()

def rssi(self):

try:

return self.wlan.status(&quot;rssi&quot;)

except:

return 0

wifi = WiFiManager()

wifi.connect()


# ===== BLYNK CLIENT =====

class BlynkClient:

BASE_URL = &quot;http://blynk.cloud/external/api&quot;

def virtual_write(self, pin, value):

url = &quot;{/update?token={} &amp; {}={}&quot;.format(

self.BASE_URL, BLYNK_AUTH, pin, value)

self._get(url)

def log_event(self, event, message):

url = &quot;{/logEvent?token={} &amp; event={} &amp; description={}&quot;.format(

self.BASE_URL, BLYNK_AUTH, event, message)
```

```
self._get(url)

def _get(self, url):

    try:

        r = urequests.get(url)

        r.close()

    except:

        logger.log(Logger.LEVEL_ERROR, &quot;Blynk HTTP failed&quot;,)

    blynk = BlynkClient()


# ===== HARDWARE =====

class Hardware:

    def __init__(self):

        self.pir = Pin(PIR_PIN, Pin.IN)

        self.buzzer = Pin(BUZZER_PIN, Pin.OUT)

        self.led = Pin(LED_PIN, Pin.OUT)

    def activate_alarm(self, silent=False):

        self.led.on()

        if not silent:

            for _ in range(3):

                self.buzzer.on()

                time.sleep(0.2)

                self.buzzer.off()
```

```
time.sleep(0.2)

def deactivate_alarm(self):

    self.led.off()

    self.buzzer.off()

hardware = Hardware()


# ===== SECURITY ENGINE =====

class SecurityEngine:

    def __init__(self):

        self.last_motion = 0

    def check_motion(self):

        if not config.data['system_armed']:

            return

        if hardware.pir.value() == 1:

            if time.time() - self.last_motion > MOTION_COOLDOWN:

                self.last_motion = time.time()

                config.data['motion_count'] += 1

                config.save()

                logger.log(Logger.LEVEL_WARN, 'Motion Detected')

                hardware.activate_alarm(config.data['silent_mode'])

                blynk.virtual_write('V0', 1)

                blynk.virtual_write('V2', config.data['motion_count'])
```

```
blynk.log_event(&quot;motion_alert&quot;, &quot;Intrusion detected&quot;)
```

```
else:
```

```
hardware.deactivate_alarm()
```

```
blynk.virtual_write(&quot;V0&quot;, 0)
```

```
security = SecurityEngine()
```

```
# ===== HEALTH MONITOR =====
```

```
class HealthMonitor:
```

```
def check_memory(self):
```

```
    free = gc.mem_free()
```

```
    logger.log(Logger.LEVEL_DEBUG, &quot;Free Memory: {}&quot;.format(free))
```

```
    if free < 5000:
```

```
        logger.log(Logger.LEVEL_ERROR, &quot;Low Memory - Restarting&quot;)
```

```
        machine.reset()
```

```
health = HealthMonitor()
```

```
# ===== GUARDIAN CARE AUTONOMOUS SAFETY NETWORK =====
```

```
wdt = WDT(timeout=8000)
```

```
# ===== SCHEDULER =====
```

```
class Scheduler:
```

```
def __init__(self):
```

```
    self.last_status = 0
```

```
self.last_wifi = 0

self.last_heartbeat = 0

self.last_memory = 0

def run(self):

    now = time.time()

    if now - self.last_status > STATUS_INTERVAL:

        self.last_status = now

        self.send_status()

    if now - self.last_wifi > WIFI_CHECK_INTERVAL:

        self.last_wifi = now

        wifi.check()

    if now - self.last_heartbeat > HEARTBEAT_INTERVAL:

        self.last_heartbeat = now

        blynk.virtual_write(&quot;V6&quot;, &quot;ONLINE&quot;,)

    if now - self.last_memory > MEMORY_CHECK_INTERVAL:

        self.last_memory = now

        health.check_memory()

    def send_status(self):

        blynk.virtual_write(&quot;V1&quot;, time.time())

        blynk.virtual_write(&quot;V5&quot;, wifi.rssi())

        logger.log(Logger.LEVEL_INFO, &quot;Status Sent&quot;,)

    scheduler = Scheduler()
```

```
# ===== DIAGNOSTICS =====

def diagnostics():logger.log(Logger.LEVEL_INFO, &quot;Firmware:
{}&quot;,.format(FIRMWARE_VERSION))

logger.log(Logger.LEVEL_INFO, &quot;Device: {}&quot;,.format(DEVICE_NAME))

logger.log(Logger.LEVEL_INFO, &quot;RSSI: {}&quot;,.format(wifi.rssi()))

logger.log(Logger.LEVEL_INFO, &quot;Motion Count:
{}&quot;,.format(config.data[&quot;motion_count&quot;]))

# ===== MAIN LOOP =====

logger.log(Logger.LEVEL_INFO, &quot;System Boot Completed&quot;,)

diagnostics()

while True:

    try:

        security.check_motion()

        scheduler.run()

        wdt.feed()

        time.sleep(1)

    except Exception as e:

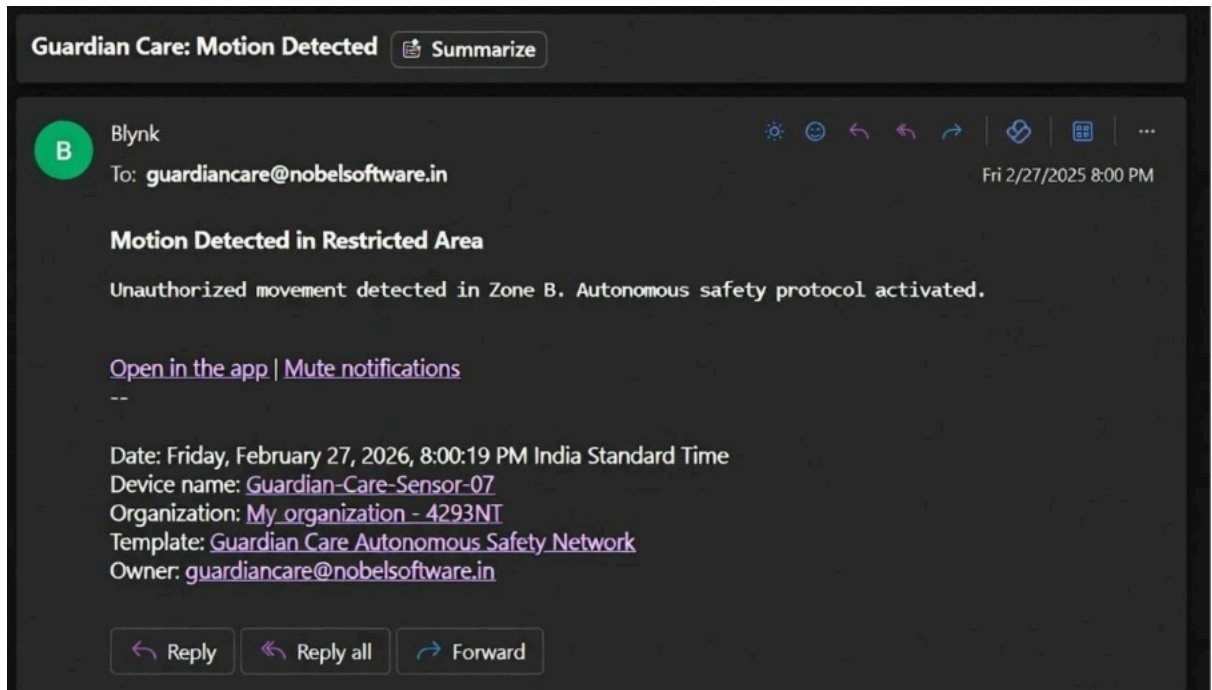
        logger.log(Logger.LEVEL_ERROR, &quot;Runtime Error: {}&quot;,.format(e))

        time.sleep(2)

# =====
```

D. SAMPLE SCREEN:

Received Mail



Form Description:

This screen shows the email received from the blynk app to the user.